네트워크 메시지 전파 시스템 분석

개념적 배경

왜 효율적인 메시지 전파가 중요한가?

Ethereum 네트워크는 수천 개의 노드가 참여하는 분산 시스템으로, 블록과 트랜잭션이 빠르고 효율적으로 전파되어야 합니다. 비효율적인 전파는 다음과 같은 문제를 야기합니다:

성능 문제:

- **블록 전파 지연**: 새 블록이 늦게 전파되면 체인 분기 증가
- **트랜잭션 지연**: 사용자 트랜잭션이 늦게 처리됨
- **네트워크 혼잡**: 불필요한 중복 전송으로 대역폭 낭비

보안 문제:

- **51% 공격 위험**: 느린 전파는 공격자에게 유리
- **MEV 기회**: 빠른 전파가 MEV 추출에 중요
- **네트워크 분할**: 일부 노드가 고립될 위험

geth의 전파 최적화 전략

계층적 전파 시스템:

1. **즉시 전파**: 작은 메시지는 직접 전송
2. **해시 기반 전파**: 큰 데이터는 해시만 전파 후 요청 시 전송
3. **적응적 대역폭 관리**: 피어별 성능에 따른 동적 조정

중복 방지 메커니즘:

- **블룸 필터**: 이미 본 트랜잭션 추적
- **피어별 상태 추적**: 각 피어가 알고 있는 정보 관리
- **지능형 라우팅**: 최적 경로로만 전파

메시지 처리 아키텍처

1. 핸들러 구조 (eth/handler.go)

```
type handler struct {
nodeID enode.ID
networkID  uint64
// 동기화 상태
snapSync atomic.Bool // snap 동기화 활성화 여부
synced atomic.Bool // 동기화 완료 여부
// 핵심 컴포넌트
database ethdb.Database
txpool txPool
chain *core.BlockChain
downloader *downloader.Downloader
txFetcher *fetcher.TxFetcher
peers *peerSet
// 이벤트 채널
txsCh  chan core.NewTxsEvent // 새 트랜잭션 이벤트
txsSub event.Subscription // 트랜잭션 구독
```

```
blockRange *blockRangeState // 블록 범위 상태

// 제어 채널

quitSync  chan  struct{}

}
```

2. 메시지 라우팅 시스템

```
graph  TB

A[수신 메시지]  -->  B{메시지 타입 확인}

B -->|트랜잭션|  C[트랜잭션 처리기]

B -->|블록|  D[블록 처리기]

B -->|상태 요청|  E[상태 처리기]

C -->  F[TxFetcher]

C -->  G[TxPool]

D -->  H[BlockFetcher]

D -->  I[Downloader]

E -->  J[SnapSync]

E -->  K[StateDB]

F -->  L[전파 결정]

G -->  L

H -->  L

L -->  M[피어 선택]

M -->  N[메시지 전송]
```

```
style A fill:#e1f5fe

style L fill:#fff3e0

style N fill:#c8e6c9
```

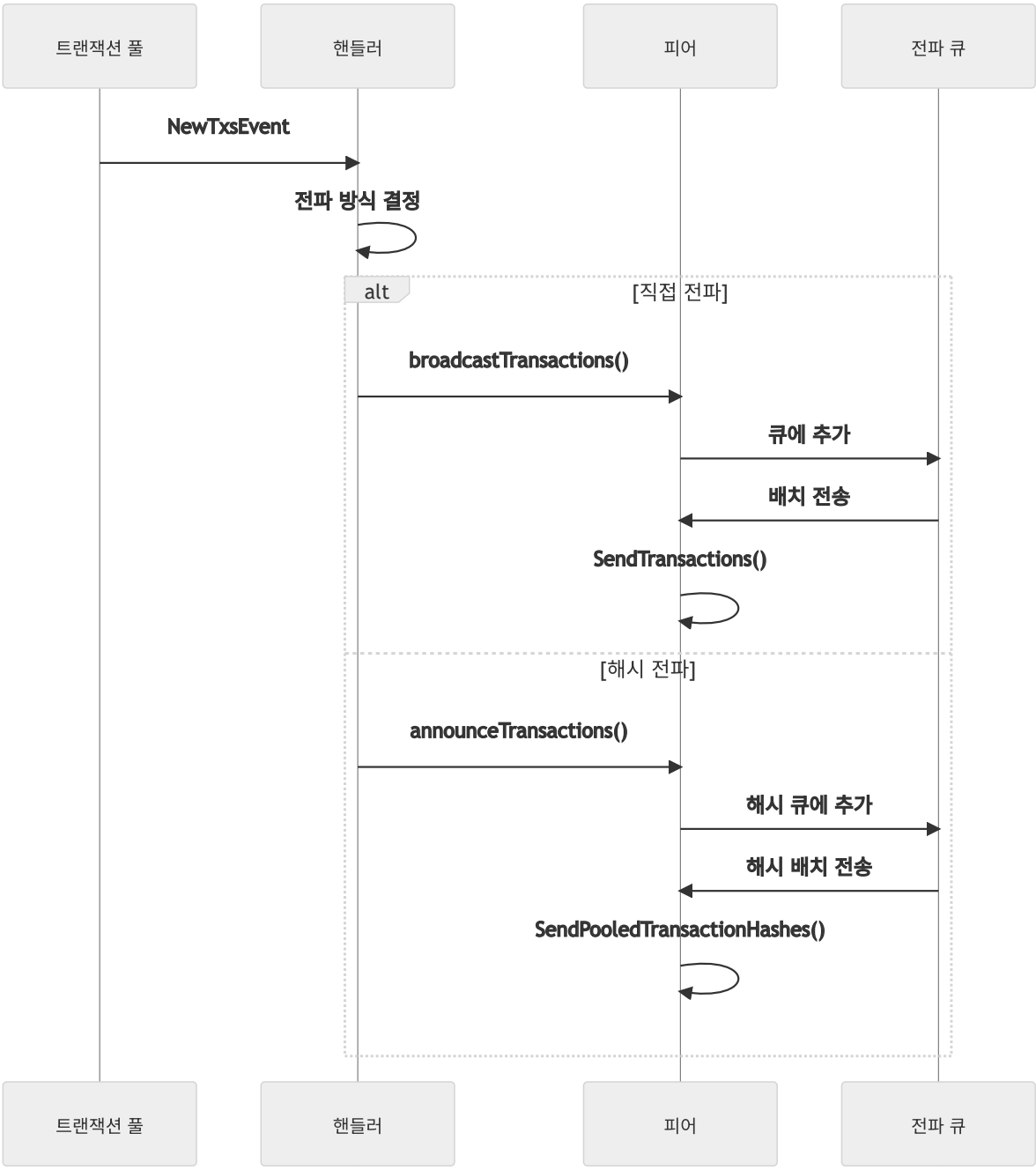
트랜잭션 전파 시스템

1. 전파 전략 구분

```go
const (

// 직접 전파할 트랜잭션 최대 크기

txMaxBroadcastSize = 4096

// 패킷 최대 크기

maxTxPacketSize = 100 * 1024

)


// 전파 방식 결정

func (h *handler) BroadcastTransactions(txs types.Transactions) {

var (

directTxs []*types.Transaction // 직접 전파할 트랜잭션들

announceTxs []common.Hash // 해시만 전파할 트랜잭션들

)

for  _, tx := range  txs {

if  tx.Size() < txMaxBroadcastSize {
```

```
directTxs = append(directTxs, tx)
} else {
announceTxs = append(announceTxs, tx.Hash())
}
}
// 피어별로 적절한 방식으로 전파
h.peers.BroadcastTransactions(directTxs)
h.peers.AnnounceTransactions(announceTxs)
}
```

2. 비동기 전파 시스템 (eth/protocols/eth/broadcast.go)


트랜잭션 풀
핸들러
피어
전파 큐
NewTxsEvent
전파 방식 결정
alt
[직접 전파]
broadcastTransactions()
큐에 추가
배치 전송
SendTransactions()
[해시 전파]
announceTransactions()
해시 큐에 추가
해시 배치 전송
SendPooledTransactionHashes()
트랜잭션 풀
핸들러
피어
전파 큐

직접 전파 구현

```go
func (p *Peer) broadcastTransactions() {
var (
queue []common.Hash // 전파할 트랜잭션 해시 큐
done chan struct{} // 전송 완료 신호
fail = make(chan error, 1) // 에러 채널
failed bool // 실패 플래그
)
for {
// 대기 중인 전송이 없고 큐에 데이터가 있으면 전송 시작
if done == nil && len(queue) > 0 {
var (
txs []*types.Transaction
size common.StorageSize
)
// 패킷 크기 제한까지 트랜잭션 수집
for i := 0; i < len(queue) && size < maxTxPacketSize; i++ {
if tx := p.txpool.Get(queue[i]); tx != nil {
txs = append(txs, tx)
size += common.StorageSize(tx.Size())
}
}
// 비동기 전송
```

```
if len(txs) > 0 {
done = make(chan struct{})
go func() {
if err := p.SendTransactions(txs); err != nil {
fail <- err
return
}
close(done)
}()
}
}
// 이벤트 처리
select {
case hashes := <-p.txBroadcast:
if !failed {
queue = append(queue, hashes...)
// 큐 크기 제한
if len(queue) > maxQueuedTxs {
queue = queue[len(queue)-maxQueuedTxs:]
}
}
case <-done:
done = nil
```

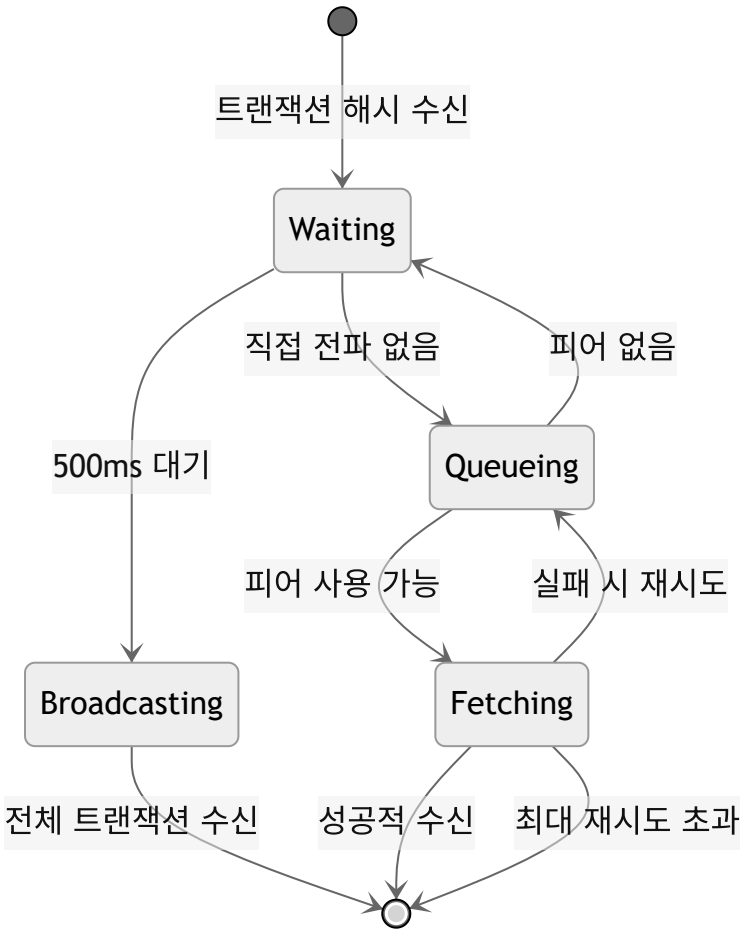
```
case <-fail:
failed = true
case <-p.term:
return
}
}
}
```

3. 트랜잭션 페처 시스템

```
type TxFetcher struct {
// 3단계 상태 관리
waitlist  map[common.Hash]map[string]struct{} // 1단계: 대기 목록
announces  map[string]map[common.Hash]*txMetadata // 2단계: 공지 목록
fetching  map[common.Hash]string  // 3단계: 페치 중
// 성능 최적화
underpriced *lru.Cache[common.Hash, time.Time] // 저가 트랜잭션 캐시
// 제어 매개변수
txSeq  uint64  // 트랜잭션 시퀀스 번호
}

const (
maxTxAnnounces = 4096  // 최대 공지 수
```

```
maxTxRetrievals = 256  // 최대 검색 수

maxTxRetrievalSize = 128 * 1024  // 최대 검색 크기

txArriveTimeout = 500 * time.Millisecond  // 도착 대기 시간

)
```

3단계 페치 프로세스



블록 전파 시스템

1. 블록 전파 전략

```
type NewBlockMsg struct {

Block *types.Block // 전체 블록 데이터
```

```
TD *big.Int // 총 난이도

}


type NewBlockHashesPacket []struct {

Hash common.Hash // 블록 해시

Number  uint64  // 블록 번호

}


// 블록 전파 결정 로직

func (h *handler) BroadcastBlock(block *types.Block, propagate  bool) {

hash := block.Hash()

peers := h.peers.PeersWithoutBlock(hash)

if  propagate {

// 일부 피어에게는 전체 블록 전송

transfer := peers[:int(math.Sqrt(float64(len(peers))))]

for  _, peer := range  transfer {

peer.AsyncSendNewBlock(block, td)

}

// 나머지 피어에게는 해시만 전송

for  _, peer := range  peers[len(transfer):] {

peer.AsyncSendNewBlockHash(block)

}

}
```

```
}
```

2. 적응적 전파 알고리즘

```
graph TD

A[새 블록 생성] --> B{피어 수 확인}

B -->|적음| C[모든 피어에게 전체 블록]

B -->|많음| D[전파 전략 선택]

D --> E[sqrt(n)개 피어에게 전체 블록]

D --> F[나머지 피어에게 해시만]

E --> G[즉시 전파]

F --> H[해시 전파]

H --> I[피어가 요청 시]

I --> J[블록 데이터 전송]

style A fill:#e1f5fe

style G fill:#c8e6c9

style J fill:#fff3e0
```

대역폭 관리 시스템

1. 메시지 레이트 추적 (p2p/msgrate/msgrate.go)

```
type Tracker struct {

// 피어별 처리 용량 (초당 아이템 수)
```

```
capacity  map[uint64]float64
// 라운드트립 시간
roundtrip time.Duration
lock sync.RWMutex
}

const (
measurementImpact = 0.1  // 측정값 영향도
capacityOverestimation = 1.01  // 용량 과대추정 비율
rttMinEstimate = 2 * time.Second  // 최소 RTT
rttMaxEstimate = 20 * time.Second  // 최대 RTT
)

// 피어 용량 계산
func (t *Tracker) Capacity(kind  uint64, targetRTT time.Duration) int {
t.lock.RLock()
defer  t.lock.RUnlock()
// 실제 측정된 처리량 계산
throughput := t.capacity[kind] * float64(targetRTT) / float64(time.Second)
// 과대추정으로 최적값 탐색
return  roundCapacity(1 + capacityOverestimation*throughput)
}
```

```
// 성능 측정값 업데이트
func (t *Tracker) Update(kind uint64, elapsed time.Duration, items int) {
t.lock.Lock()
defer t.lock.Unlock()
if items == 0 {
// 타임아웃 또는 데이터 없음 → 용량 0으로 설정
t.capacity[kind] = 0
return
}
// 새로운 측정값으로 용량 업데이트
measured := float64(items) / (float64(elapsed) / float64(time.Second))
t.capacity[kind] = (1-measurementImpact)*t.capacity[kind] + measurementImpact*me
t.roundtrip = time.Duration((1-measurementImpact)*float64(t.roundtrip) + measure
}
```

2. 동적 요청 크기 조정

```
graph LR
A[요청 전송] --> B[응답 시간 측정]
B --> C{성능 평가}
C -->|빠름| D[요청 크기 증가]
C -->|느림| E[요청 크기 감소]
```

```
C -->|적정|  F[현재 크기 유지]

D -->  G[다음 요청]

E -->  G

F -->  G

G -->  A

style C fill:#fff3e0

style D fill:#c8e6c9

style E fill:#ffcdd2
```

3. 피어 품질 평가

```go
type PeerQuality struct {

responseTime time.Duration // 평균 응답 시간

successRate  float64  // 성공률

bandwidth  uint64  // 대역폭

reliability  float64  // 신뢰도

}


// 피어 품질 업데이트

func (p *Peer) updateQuality(responseTime time.Duration, success  bool, dataSize

// 지수 이동 평균으로 응답 시간 업데이트

p.quality.responseTime = time.Duration(

0.9*float64(p.quality.responseTime) + 0.1*float64(responseTime),
```

```
)

// 성공률 업데이트

if  success {

p.quality.successRate = p.quality.successRate*0.95 + 0.05

} else {

p.quality.successRate = p.quality.successRate * 0.95

}

// 대역폭 계산

if  responseTime > 0 {

currentBandwidth := dataSize * uint64(time.Second) / uint64(responseTime)

p.quality.bandwidth = uint64(0.9*float64(p.quality.bandwidth) + 0.1*float64(curr

}

}



// 피어 선택 알고리즘

func (ps *peerSet) selectBestPeers(count  int, msgType  uint64) []*Peer {

peers := ps.AllPeers()

// 품질 점수로 정렬

sort.Slice(peers, func(i, j  int) bool {

scoreI := calculatePeerScore(peers[i], msgType)

scoreJ := calculatePeerScore(peers[j], msgType)

return  scoreI > scoreJ

})
```

```
if len(peers) > count {
peers = peers[:count]
}
return peers
}

func calculatePeerScore(peer *Peer, msgType uint64) float64 {
capacity := peer.tracker.Capacity(msgType, 2*time.Second)
rtt := float64(peer.tracker.roundtrip) / float64(time.Second)
// 용량이 높고 RTT가 낮을수록 높은 점수
return float64(capacity) / (1 + rtt)
}
```

중복 방지 및 최적화

1. 블룸 필터 기반 중복 방지

```
type announceData struct {
hash common.Hash
number uint64
header *types.Header
time time.Time
}
```

```go
// 피어별 알려진 데이터 추적
type knownCache struct {
hashes *lru.Cache[common.Hash, time.Time]
blocks *lru.Cache[common.Hash, time.Time]
txs *lru.Cache[common.Hash, time.Time]
}

func (p *Peer) MarkTransaction(hash common.Hash) {
p.knownTxs.Add(hash, time.Now())
}

func (p *Peer) MarkBlock(hash common.Hash) {
p.knownBlocks.Add(hash, time.Now())
}

// 중복 확인
func (p *Peer) KnowsTransaction(hash common.Hash) bool {
_, known := p.knownTxs.Get(hash)
return  known
}
```

2. 지능형 피어 선택

```
// 메시지 타입별 최적 피어 선택
func (h *handler) selectPeersForBroadcast(msgType string, data interface{}) []
switch msgType {
case "transaction":
// 트랜잭션: 빠른 전파를 위해 많은 피어 선택
return h.peers.BestPeers(16, eth.TransactionsMsg)
case "block":
// 블록: 네트워크 토폴로지 고려한 선택적 전파
return h.selectBlockPropagationPeers(data.(*types.Block))
case "announcement":
// 공지: 모든 연결된 피어에게 전송
return h.peers.AllPeers()
}
return nil
}

func (h *handler) selectBlockPropagationPeers(block *types.Block) []*Peer {
allPeers := h.peers.PeersWithoutBlock(block.Hash())
// 네트워크 지연 기반 계층적 전파
var (
fastPeers []*Peer // 빠른 피어들
normalPeers []*Peer // 일반 피어들
```

```
)

for _, peer := range allPeers {

if peer.tracker.roundtrip < 100*time.Millisecond {

fastPeers = append(fastPeers, peer)

} else {

normalPeers = append(normalPeers, peer)

}

}

// 빠른 피어들에게 우선 전파

selected := fastPeers

if len(selected) < 8 {

needed := 8 - len(selected)

if needed > len(normalPeers) {

needed = len(normalPeers)

}

selected = append(selected, normalPeers[:needed]...)

}

return selected

}
```

3. 메모리 최적화

```
// 순환 버퍼를 사용한 메모리 효율적 큐
```

```
type circularQueue struct {
items []interface{}
head int
tail int
size int
cap int
}

func (q *circularQueue) Push(item interface{}) {
if q.size == q.cap {
// 큐가 가득 찬 경우 가장 오래된 항목 제거
q.head = (q.head + 1) % q.cap
q.size--
}
q.items[q.tail] = item
q.tail = (q.tail + 1) % q.cap
q.size++
}

// 가비지 컬렉션 압박 감소를 위한 객체 풀
var messagePool = sync.Pool{
New: func() interface{} {
return &Message{
```

```
data: make([]byte, 0, 1024),

}

},

}


func  getMessage() *Message {

return  messagePool.Get().(*Message)

}


func  putMessage(msg *Message) {

msg.Reset()

messagePool.Put(msg)

}
```

성능 모니터링 및 메트릭

1. 전파 성능 메트릭

```
var (

// 트랜잭션 전파 메트릭

txBroadcastTimer = metrics.NewRegisteredTimer("eth/handler/tx/broadcast", nil)

txAnnounceTimer = metrics.NewRegisteredTimer("eth/handler/tx/announce", nil)

txPropagationGauge = metrics.NewRegisteredGauge("eth/handler/tx/propagation", ni
```

```
// 블록 전파 메트릭
blockBroadcastTimer = metrics.NewRegisteredTimer("eth/handler/block/broadcast",
blockPropagationGauge = metrics.NewRegisteredGauge("eth/handler/block/propagatio
// 피어 성능 메트릭
peerResponseTimer = metrics.NewRegisteredTimer("eth/handler/peer/response", nil)
peerBandwidthGauge = metrics.NewRegisteredGauge("eth/handler/peer/bandwidth", ni
)


// 전파 시간 측정
func (h *handler) measurePropagationTime(msgType  string, hash common.Hash) {
start := time.Now()
// 전파 완료 콜백
h.onPropagationComplete(hash, func() {
elapsed := time.Since(start)
switch  msgType {
case  "transaction":
txPropagationGauge.Update(elapsed.Nanoseconds())
case  "block":
blockPropagationGauge.Update(elapsed.Nanoseconds())
}
})
}
```

2. 실시간 성능 대시보드

```
type PropagationStats struct {
TxPropagationTime time.Duration `json:"tx_propagation_time"`
BlockPropagationTime time.Duration `json:"block_propagation_time"`
ActivePeers int `json:"active_peers"`
AverageBandwidth uint64 `json:"average_bandwidth"`
MessageQueueSize int `json:"message_queue_size"`
}

func (h *handler) GetPropagationStats() *PropagationStats {
return &PropagationStats{
TxPropagationTime: h.getAverageTxPropagationTime(),
BlockPropagationTime: h.getAverageBlockPropagationTime(),
ActivePeers: h.peers.Len(),
AverageBandwidth: h.getAveragePeerBandwidth(),
MessageQueueSize: h.getMessageQueueSize(),
}
}
```

실제 사용 예시

---

1. 트랜잭션 전파 설정

```
// 핸들러 초기화
config := &handlerConfig{
NodeID: nodeID,
Database: database,
Chain: blockchain,
TxPool: txpool,
Network: networkID,
}

handler, err := newHandler(config)
if err != nil {
log.Fatal("Failed to create handler:", err)
}

// 트랜잭션 이벤트 구독
txsCh := make(chan core.NewTxsEvent, txChanSize)
handler.txsSub = handler.txpool.SubscribeTransactions(txsCh, true)

// 전파 루프 시작
go handler.txBroadcastLoop()
```

2. 커스텀 전파 정책

```
type CustomPropagationPolicy struct {
maxPeers  int
minConfidence  float64
priorityGasPrice *big.Int
}


func (p *CustomPropagationPolicy) ShouldPropagate(tx *types.Transaction, peer *P
// 가스 가격 기반 우선순위
if  tx.GasPrice().Cmp(p.priorityGasPrice) >= 0 {
return true
}
// 피어 신뢰도 확인
if  peer.quality.reliability < p.minConfidence {
return false
}
// 피어 수 제한
return  len(peer.knownTxs.Keys()) < p.maxPeers
}
```

3. 성능 최적화 설정

```
// 전파 매개변수 튜닝
```

```
const (
maxTxBroadcastPeers = 16  // 트랜잭션 직접 전파 피어 수
maxTxAnnouncePeers = 64  // 트랜잭션 해시 전파 피어 수
maxBlockBroadcastPeers = 8  // 블록 직접 전파 피어 수
txBroadcastInterval = 100 * time.Millisecond  // 전파 간격
blockBroadcastTimeout = 2 * time.Second  // 블록 전파 타임아웃
)


// 동적 매개변수 조정
func (h *handler) adjustPropagationParams() {
networkLoad := h.getNetworkLoad()
peerCount := h.peers.Len()
if  networkLoad > 0.8 {
// 네트워크 부하가 높으면 전파 피어 수 감소
h.maxBroadcastPeers = max(4, h.maxBroadcastPeers/2)
} else  if  networkLoad < 0.3 && peerCount > 20 {
// 네트워크 부하가 낮으면 전파 피어 수 증가
h.maxBroadcastPeers = min(32, h.maxBroadcastPeers*2)
}
}
```

문제 해결 가이드

일반적인 문제들

1. 전파 지연 문제

증상: 트랜잭션이나 블록이 늦게 전파됨

원인:

- 피어 연결 수 부족
- 네트워크 대역폭 제한
- 피어 품질 저하

해결방법:

- MaxPeers 설정 증가
- 네트워크 연결 최적화
- 저품질 피어 제거

2. 메모리 사용량 증가

증상: 메모리 사용량이 계속 증가

원인:

- 메시지 큐 크기 제한 없음
- 가비지 컬렉션 압박
- 메모리 누수

해결방법:

- 큐 크기 제한 설정
- 객체 풀 사용

```
- 정기적 메모리 정리
```

3. 네트워크 혼잡

```
증상: 네트워크 대역폭 포화

원인:

- 과도한 중복 전송

- 비효율적 피어 선택

- 대역폭 제한 없음

해결방법:

- 중복 방지 메커니즘 강화

- 지능형 피어 선택

- 대역폭 제한 설정
```

성능 튜닝 가이드

```
// 고성능 설정

config := PropagationConfig{

MaxTxBroadcastPeers: 32,

MaxBlockBroadcastPeers: 16,

TxBroadcastInterval: 50 * time.Millisecond,

BlockBroadcastTimeout: 1 * time.Second,

// 메모리 최적화

MaxQueueSize: 10000,
```

```
MaxKnownTxs: 32768,

MaxKnownBlocks: 1024,

// 대역폭 관리

MaxBandwidthPerPeer: 10 * 1024 * 1024, // 10MB/s

MinPeerQuality: 0.8,

}
```

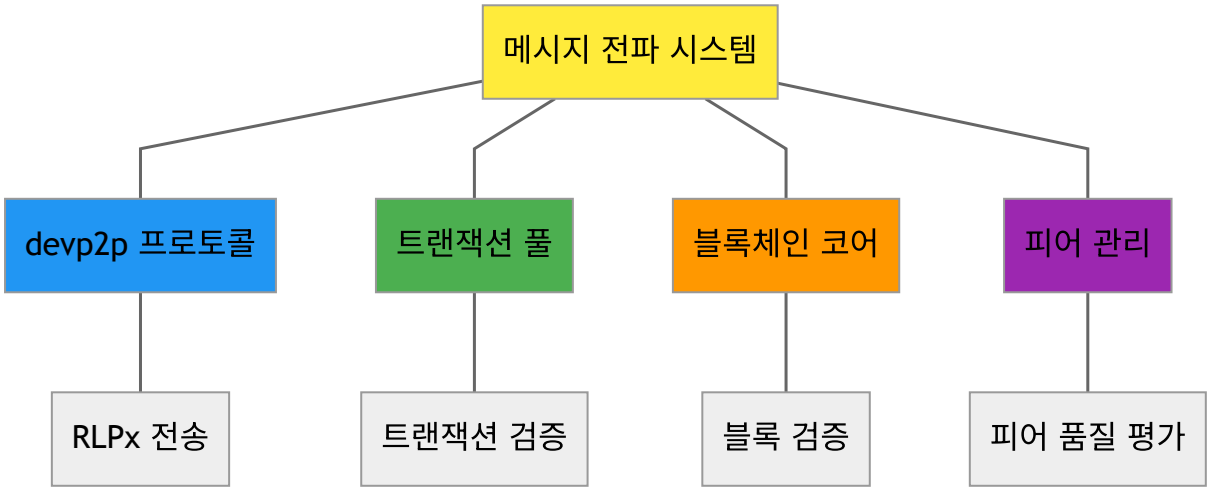
모니터링 설정

```
// 메트릭 수집 설정

metrics.Enabled = true


// 주요 모니터링 지표

monitoringMetrics := []string{

"eth/handler/tx/broadcast",

"eth/handler/block/propagation",

"eth/handler/peer/bandwidth",

"eth/handler/queue/size",

}


// 알림 임계값 설정

alertThresholds := map[string]float64{

"tx_propagation_time": 1000, // 1초
```

```
"block_propagation_time": 2000, // 2초

"peer_response_time": 5000, // 5초

}
```

관련 시스템

의존성 관계



영향 범위

직접 영향:

- 네트워크 전체 성능
- 사용자 경험 (트랜잭션 확인 시간)
- 노드 간 동기화 속도

간접 영향:

- MEV 기회 및 공정성
- 네트워크 보안 수준
- 에너지 효율성

이 분석을 통해 geth의 네트워크 메시지 전파 시스템이 어떻게 효율적이고 확장 가능한 방식으로 블록과 트랜잭션을 전파하는지, 그리고 대역폭 관리와 피어 평가 시스템이 어떻게 네트워크 성능을 최적화하는지 완전히 이해할 수 있습니다.